📝 **Java Exceptions – Notes**

1⃣ What is an Exception?

- An **exception** is an event that occurs during program execution that **disrupts normal flow**.
- Example: divide by zero, null object access, file not found.
- All exceptions are objects derived from **Throwable**.

2⃣ Exception Hierarchy

```
Throwable
│
├── Error (serious, usually not handled)
│   ├── OutOfMemoryError
│   ├── StackOverflowError
│   └── VirtualMachineError
│
└── Exception (can be handled)
    │
    ├── Checked Exceptions (must handle)
    │   ├── IOException
    │   │   └── FileNotFoundException
    │   ├── SQLException
    │   └── ClassNotFoundException
    │
    └── Unchecked Exceptions (RuntimeException)
        ├── ArithmeticException
        ├── NullPointerException
        ├── ArrayIndexOutOfBoundsException
        ├── IllegalArgumentException
        └── NumberFormatException
```

3⃣ Checked vs Unchecked

Type	Compiler Behavior	Examples
Checked	Must handle (try-catch) or declare	IOException, SQLException
Unchecked	Compiler doesn't force handling	NullPointerException, ArithmeticException
Error	Serious JVM issues	OutOfMemoryError, StackOverflowError

---

4⃣ Handling Exceptions

try-catch-finally

```
try {
   // code that may throw an exception
} catch (ExceptionType e) {
   // handle exception
} finally {
   // optional, always executed
}
```

Notes:

- Multiple catch blocks allowed for different exception types.
- finally runs **always**, used for resource cleanup.
- **Try-with-resources**: automatically closes resources like files.

5⃣ Example: Checked & Unchecked

```
// Checked Exception
```

```
try (BufferedReader reader = new BufferedReader(new FileReader("file.txt"))) {
   System.out.println(reader.readLine());
} catch (FileNotFoundException e) {
   System.out.println("File not found!");
} catch (IOException e) {
   System.out.println("Error reading file!");
}

// Unchecked Exception
try {
   int x = 10 / 0; // ArithmeticException
} catch (ArithmeticException e) {
   System.out.println("Divide by zero!");
}
```

6️⃣ Best Practices

- Always check for **null or invalid inputs**.
- Prefer **try-with-resources** for files/DB connections.
- Handle exceptions **specifically**, not just Exception.
- Include **meaningful messages** in catch.

Why are RuntimeExceptions unchecked?
RuntimeException‌s are unchecked because they represent programming errors like null access or invalid arguments. The compiler doesn't force handling them because they should be fixed in code logic rather than caught at runtime.

Why is IOException checked but NullPointerException unchecked?
Now we connect the dots:

- IOException happens due to **external conditions** (file missing, network down, disk issue).
- You **cannot fully prevent it by logic**.
- So the compiler forces you to **acknowledge and handle it**.

Whereas:

- NullPointerException happens due to **bad code logic**.
- You *can* prevent it by proper null checks.
- So the compiler does NOT force you.

👉 Why is IOException checked but NullPointerException unchecked?
IOException is checked because it occurs due to external factors like file or network failures, which cannot be fully prevented by program logic. Therefore, the compiler forces us to handle it.
NullPointerException is unchecked because it results from programming mistakes, and it should be prevented through proper coding practices rather than mandatory handling.

🧠 **Quick Reinforcement (Very Important)**
Tell me:
If I write:

```
public void read() throws IOException {
  // file reading logic
}
```

Am I handling the exception or just passing responsibility?
👉 One line answer.

💬 **Strong Interview Answer**
In layered architecture, checked exceptions are usually propagated upward and handled at the top layer (like Controller or Global Exception Handler) to maintain separation of concerns and centralized error handling.

⚡ Custom Exceptions
So if a user has insufficient balance, should we throw:

- IOException ❌
- NullPointerException ❌
- Or something meaningful like InsufficientBalanceException ✅

Exactly.

💡 **Why We Create Custom Exceptions**
We create custom exceptions when:

1. It represents a **business rule violation**
2. We want **clear, meaningful error names**
3. We want better **readability & maintainability**
4. We want centralized handling (e.g., map to proper HTTP status)

If you create:
public class InsufficientBalanceException extends RuntimeException
Why would you extend RuntimeException instead of Exception?
Why extend RuntimeException for business exceptions?
Because:

- It represents a **business validation failure**
- We usually want it to **bubble up to the controller**
- We handle it centrally (e.g., @ControllerAdvice)
- We don't want to force every service method to write unnecessary try-catch

---

🔥 **Strong Interview Answer**
We extend RuntimeException for business exceptions because they represent validation or domain rule failures. These should propagate to the top layer and be handled centrally rather than forcing every method in the call stack to catch or declare them.

If I extend Exception instead of RuntimeException, what changes behavior-wise?
If you write:
public class InsufficientBalanceException extends Exception
What will the compiler now force you to do in the withdraw() method?

If you extend:
public class InsufficientBalanceException extends Exception
Then it becomes a **checked exception**.
And what do checked exceptions require?
👉 The compiler WILL force you to either:

- Handle it using try-catch
  OR
- Declare it using throws

So your withdraw() method would need:
public void withdraw(double amount) throws InsufficientBalanceException
And every caller must also handle or declare it.

🔥 **So the real difference:**

Extends	Type	Compiler Forces Handling?
Exception	Checked	✅ Yes
RuntimeException	Unchecked	❌ No

📘 **Notes: throw vs throws**

✅ **1️⃣ throw**

- Used **inside method body**
- Actually throws an exception object
- Used with new
- Happens at **runtime**

Example:

throw new IllegalArgumentException("Invalid amount");

Interview Q & A

Q: What does throw do?

It is used to explicitly throw an exception object at runtime.

Q: Can we throw checked and unchecked exceptions using throw?

Yes, both can be thrown using throw.

✅ **2️⃣ throws**

- Used in **method signature**
- Declares that method may throw an exception
- Transfers responsibility to caller
- Mostly used with **checked exceptions**

Example:

public void readFile() throws IOException

Interview Q & A

Q: What does throws do?

It declares that a method may throw an exception and passes handling responsibility to the caller.

Q: Is throws mandatory for RuntimeException?

No, because RuntimeExceptions are unchecked.

Q: Can we still write throws for RuntimeException?

Yes, it's optional and improves readability.

🔥 **Difference Table**

Feature	throw	throws
Used where	Inside method	Method signature
Purpose	Actually throw exception	Declare/propagate exception
Time	Runtime	Compile-time declaration
Multiple allowed	Only one at a time	Can declare multiple

📘 **Notes: Why We Extend RuntimeException in Business Apps**

✅ **Core Idea**

Business exceptions usually represent:

- Validation failures
- Domain rule violations
- Application logic errors

Example:

- InsufficientBalanceException
- OrderAlreadyCancelledException
- UserNotAuthorizedException

🔥 **Why extend RuntimeException?**

1️⃣ They represent programming/business rule failures

They are not external system failures like IOException.

2️⃣ We don't want to force every layer to handle them

In layered architecture:
Controller → Service → Repository
If business exceptions were checked:

- Every method would need throws
- Code becomes cluttered

3️⃣ We handle them centrally
Using:

- @ControllerAdvice
- Global exception handlers

🎯 **Interview-Level Answer**
We extend RuntimeException for business exceptions because they represent validation or domain rule failures. These should propagate to the top layer and be handled centrally instead of forcing every method in the call stack to catch or declare them.

📌 **Important Conceptual Q & A**
Q: If we extend Exception instead of RuntimeException, what happens?
It becomes a checked exception and the compiler forces handling or declaration.
Q: Is it still a custom exception if it extends Exception?
Yes. It is still custom, but it becomes checked.
Q: Why are RuntimeExceptions unchecked?
Because they represent programming mistakes that should be fixed in logic rather than being forced to handle everywhere.

🧠 **Ultra-Short Memory Hook**

- throw → action
- throws → declaration
- Business exceptions → extend RuntimeException
- External failures → checked exceptions

1️⃣ finally Block – Deep Behavior
✅ **What is finally?**

- Executes **always**, whether:
  - Exception occurs or not
  - Exception is handled or not
  - return is used inside try or catch

🔥 **Important Behaviors**
✅ **1. finally runs even after return**

```
public int test() {
  try {
    return 10;
  } finally {
    System.out.println("Finally executed");
  }
}
```

Output:
Finally executed
✓ JVM stores return value first
✓ Then executes finally
✓ Then returns

⚠ **2. If finally also has return**

```
public int test() {
  try {
    return 10;
  } finally {
    return 20;
  }
}
```

Result → 20

⚠ Finally overrides previous return

⚠ Very bad practice

❌ **When finally does NOT execute**

- If JVM crashes
- If System.exit(0) is called
- Fatal JVM error (e.g., OutOfMemoryError)

💬 **Interview Line**

The finally block executes regardless of exception occurrence, except in cases like JVM shutdown or fatal errors.

2️⃣ Try-With-Resources Internals

Introduced in **Java 7**

✅ **Purpose**

Automatically closes resources that implement:

AutoCloseable

Example:

- File streams
- DB connections
- BufferedReader

Example

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
  System.out.println(br.readLine());
}
```

No need for finally block.

🔥 **Internal Behavior**

Compiler converts it into:

- try block
- finally block
- resource.close()

Automatically calls:

resource.close();

Important Rule

Resources are closed in **reverse order** of declaration.

💬 **Interview Line**

Try-with-resources ensures automatic resource management by internally generating a finally block that calls close() on AutoCloseable resources.

3️⃣ Exception Chaining

✅ **What is it?**

Wrapping one exception inside another.
Used when:

- Lower layer throws exception
- Upper layer wants to throw meaningful business exception

Example

```
try {
   // DB code
} catch (SQLException e) {
   throw new DataAccessException("Database error", e);
}
```

Here:

- DataAccessException is outer
- SQLException is cause

Why Important?

- Preserves root cause
- Improves debugging
- Keeps abstraction clean

Get Cause

```
e.getCause();
```

💬 **Interview Line**

Exception chaining preserves the original cause while throwing a higher-level abstraction exception.

4️⃣ Overriding Rules with Exceptions

Very important interview topic.

Rule 1️⃣ – Unchecked exceptions

Subclass can throw:

- Same unchecked
- New unchecked
- More unchecked

✓ No restriction

Rule 2️⃣ – Checked exceptions

Subclass:

- Cannot throw broader checked exception
- Can throw:
  - Same exception
  - Narrower (subclass) exception

Example

Parent:

```
public void read() throws IOException
```

Child can:

```
public void read() throws FileNotFoundException  // ✓ allowed
```

Child cannot:

```
public void read() throws Exception  // ❌ not allowed
```

Rule 3️⃣ If parent doesn't throw checked exception

Child cannot introduce new checked exception.

💬 **Interview Line**
Overriding methods cannot throw broader checked exceptions than the parent method.

5️⃣ Multi-Catch & Rethrow

✅ **Multi-Catch (Java 7+)**

```
try {
  // code
} catch (IOException | SQLException e) {
  System.out.println("Handled");
}
```

Rules:

- Exceptions must not have parent-child relationship
- Exception variable is implicitly final

✅ **Rethrow**

```
catch (IOException e) {
   throw e;
}
```

You can rethrow same exception.

Java 7 Improved Rethrow
Compiler tracks exact exception type when rethrowing.

💬 **Interview Line**
Multi-catch improves readability by handling multiple unrelated exceptions in a single catch block.

6️⃣ Suppressed Exceptions

Very advanced topic 🔥
Occurs in **try-with-resources**.

What Happens?
If:

- Exception occurs in try block
- AND another exception occurs during close()

The close() exception becomes **suppressed**.

Example

```
try (MyResource r = new MyResource()) {
   throw new RuntimeException("Main exception");
}
```

If close() throws exception:

- Main exception is thrown
- Close exception becomes suppressed

How to Get Suppressed Exceptions

```
Throwable[] suppressed = e.getSuppressed();
```

Why Important?

- Prevents losing original exception
- Keeps debugging accurate

💬 **Interview Line**

Suppressed exceptions occur when both the try block and resource closing throw exceptions; the closing exception is attached to the primary exception.

🧠 Java Exceptions – 1-Minute Memory Cheat

🔹 1️⃣ Hierarchy

```
Throwable
├── Error (JVM issues – don't handle)
└── Exception
    ├── Checked (compiler forces handling)
    └── RuntimeException (unchecked)
```

- Checked → External problems (IOException)
- Unchecked → Programming mistakes (NullPointerException)
- Error → JVM failure (OutOfMemoryError)

🔹 2️⃣ throw vs throws

- throw → actually throws exception (inside method)
- throws → declares exception (method signature)

✓ throw = action

✓ throws = declaration

🔹 3️⃣ Checked vs Unchecked Logic

- Checked → Cannot prevent fully by logic → must handle
- Unchecked → Programming mistake → fix logic

🔹 4️⃣ finally Behavior

- Always executes
- Executes even after return
- Doesn't execute if:
  - System.exit()
  - JVM crash

⚠ Avoid return inside finally

🔹 5️⃣ Try-With-Resources

- Auto closes resources
- Resource must implement AutoCloseable
- Closes in reverse order
- Internally converted to try-finally

🔹 6️⃣ Custom Exceptions

- Used for business rule violations
- Usually extend RuntimeException
- Handled at controller/global level

🔹 7️⃣ Overriding Rules

- Cannot throw broader checked exception
- Can throw narrower checked exception
- Unchecked → no restriction

🔹 8️⃣ Exception Chaining

- Wrap lower-level exception
- Preserve root cause
- Use constructor with cause

🔹 9️⃣ **Multi-Catch**

catch (IOException | SQLException e)

- No parent-child relationship allowed

🔹 🔟 **Suppressed Exceptions**

- Happens in try-with-resources
- If both try and close throw exception
- Closing exception becomes suppressed
- Retrieve using getSuppressed()

🎯 **Ultimate Interview One-Liner**

Exceptions in Java are categorized as checked, unchecked, and errors. Checked exceptions represent external failures, unchecked represent programming mistakes, and best practice involves centralized handling, try-with-resources, and meaningful custom exceptions.